S1. Tarea. Exploradores NoSQL

Ajuste y ampliación del modelo documental de Hospitalización — Base de Datos 2, Unidad 3

Estudiante: Anderson Cañas Alcaraz

Programa: Tecnología en Desarrollo de Software — Semestre 4

Motor de base de datos: MongoDB (documental), base `hce_hospitalizacion_grupo_6`

Contexto narrativo: Sistema de hospitalización — Secretaría de Salud de Antioquia

Modalidad de trabajo: Individual (sin equipo asignado)

1. Introducción y continuidad con la Unidad 2

Este trabajo continúa la actividad de la Unidad 2 ("Construyendo una base de datos NoSQL"), donde se diseñó e implementó el subsistema de Hospitalización de una Historia Clínica Electrónica usando MongoDB, con las colecciones `pacientes`, `hospitalizaciones` (que embebe `diagnosticos` y `alta`), `notas_evolucion`, `signos_vitales`, `ordenes_medicamento` y `notas_enfermeria`.

La Unidad 3 pide ampliar ese modelo cubriendo explícitamente dos entidades que antes no existían como tales: **médicos** y **camas**. No se creó una base nueva ni se borró nada de lo anterior: se trabajó sobre la misma base `hce_hospitalizacion_grupo_6`, agregando lo que faltaba.

> **Nota sobre el trabajo en equipo:** la nomenclatura del curso sigue hablando de "equipos", pero esta actividad, igual que la anterior, se desarrolló de forma individual porque no se asignaron compañeros de grupo. Ver `division_trabajo.md`.

1.1 Tecnología elegida

Se mantiene **MongoDB** (modelo documental) en vez de un modelo de grafos (Neo4j), por la misma razón que en la Unidad 2: la información de una hospitalización es naturalmente jerárquica (un paciente tiene hospitalizaciones, una hospitalización tiene diagnósticos, notas, signos vitales y órdenes), y ese tipo de relación "contiene / pertenece a" se modela de forma muy natural con documentos y referencias, sin necesitar la flexibilidad de un grafo para relaciones muchos-a-muchos complejas.

2. Diseño del modelo: qué se ajustó

2.1 Qué faltaba y qué se agregó

Entidad	Unidad 2	Unidad 3 (este trabajo)
Médicos	Solo existía como texto suelto (ej. `medico_tratante: "Dr. Andres Felipe Suarez"`) dentro de otras colecciones.	Colección propia `medicos`, referenciada desde `hospitalizaciones`, `ordenes_medicamento` y `notas_evolucion` mediante `medico_id`.
Camas	No existía ningún concepto de cama; solo el campo `servicio` (texto).	Colección propia `camas`, referenciada desde `hospitalizaciones` mediante `cama_id`, con estado `libre` / `ocupada` / `mantenimiento`, piso y servicio.

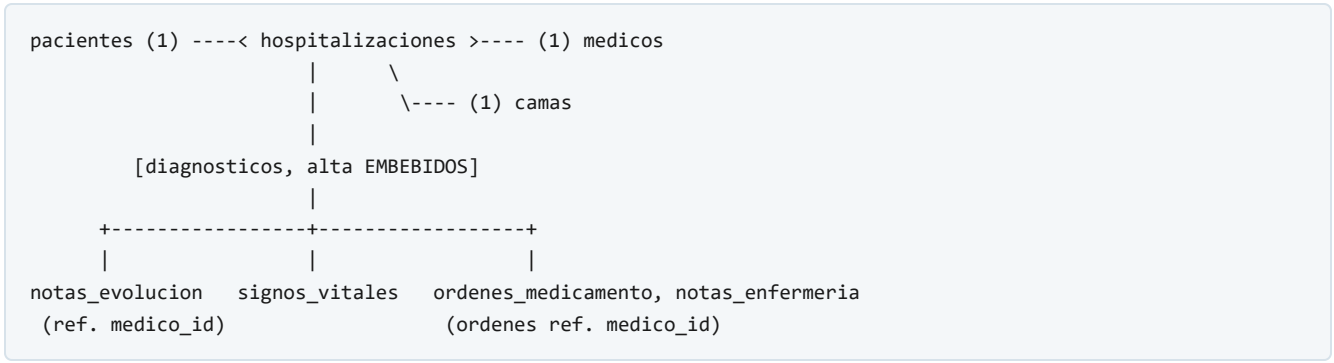
Los campos de texto originales (`medico_tratante`, `medico`, `medico_ordena`) **no se eliminaron**: se conservan como una "foto" histórica del nombre del médico en el momento del registro, mientras que el nuevo campo `*_id` se usa para las consultas en vivo. Esto evita romper nada de lo ya entregado en la Unidad 2.

2.2 Justificación: embeber vs. referenciar

Colección	Estrategia	Por qué
`medicos`	Referenciada (coleccion propia)	Un mismo médico aparece en decenas o cientos de hospitalizaciones, órdenes y notas a lo largo del tiempo. Embeberlo duplicaría su nombre, especialidad y registro profesional en cada uno de esos documentos, y cualquier cambio (ej. de teléfono) obligaría a actualizar todos esos duplicados.
`camas`	Referenciada (coleccion propia)	Una cama se reutiliza con pacientes distintos constantemente. Además, el hospital necesita poder preguntar "¿cuántas camas libres hay en Medicina Interna ahora mismo?" de forma instantánea, algo que solo es rápido si `camas` es su propia colección con su propio campo `estado`, en vez de tener que recorrer todas las hospitalizaciones para calcularlo.
`camas.hospitalizacion_actual_id`	Denormalización controlada	Este dato técnicamente se podría calcular buscando en `hospitalizaciones` con `estado:"activa"` y `cama_id` igual al de la cama, pero esa búsqueda se haría todo el tiempo. Guardar una copia controlada hace esa consulta instantánea, a cambio de que la aplicación deba mantenerla sincronizada en cada ingreso o alta (ver riesgo discutido en la reflexión ética).

El esquema completo, con el detalle de cada colección y cada decisión, está documentado en `schema.json`.

2.3 Diagrama conceptual (texto)



3. Implementación

3.1 Creación de las colecciones nuevas

Script: `01_create_medicos_camas.js`. Crea `medicos` y `camas` con validación `$jsonSchema` e índices (`registro_medico` único en médicos, `numero_cama` único en camas), y actualiza con `collMod` el validador de `hospitalizaciones`, `ordenes_medicamento` y `notas_evolucion` para documentar oficialmente el nuevo campo `medico_id` / `cama_id`.

[CAPTURA: salida de 01_create_medicos_camas.js mostrando "Colecciones creadas" y db.getCollectionNames()]

3.2 Carga de datos simulados

Script: `02_insert_medicos_camas.js`. Carga 7 médicos (los 5 que ya existían como texto en la Unidad 2, más 2 nuevos sin pacientes asignados todavía) y 16 camas repartidas en 4 servicios (UCI, Cirugía General, Medicina Interna, Ortopedia y Traumatología).

[CAPTURA: salida de 02_insert_medicos_camas.js — "7 medicos insertados" / "16 camas insertadas"]

3.3 Vinculación de los datos existentes

Script: `03_vincular_existentes.js`. Conecta las 10 hospitalizaciones que ya existían con su médico real (por coincidencia de nombre) y con una cama real del mismo servicio, y actualiza el estado de esas camas según si la hospitalización sigue activa o ya tuvo alta.

[CAPTURA: salida de 03_vincular_existentes.js mostrando las hospitalizaciones activas y sus camas ocupadas]

3.4 CRUD completo

Script: `04_crud.js`. Todas las operaciones se ejecutaron contra la base real y fueron verificadas.

Operación	Qué hace
CREATE	Inserta un paciente nuevo, su hospitalización (ya con `medico_id` y `cama_id` desde el inicio) y una orden de medicamento.
UPDATE simple	Cambia el teléfono de un paciente (campo de primer nivel).

UPDATE anidado (objeto)	Cambia `contacto_emergencia.telefono` con notación de punto, sin tocar el resto del subdocumento.
UPDATE anidado (arreglo)	Agrega un diagnóstico secundario al arreglo `diagnosticos` con `$push`.
UPDATE anidado (subdocumento)	Llena por primera vez el subdocumento `alta` de una hospitalización que estaba activa.
SOFT DELETE	Libera una cama (`estado: "libre"`) y marca un paciente como `activo: false` (baja lógica, por obligación legal de conservar la historia clínica).
HARD DELETE	Inserta y luego elimina para siempre (`deleteOne`) un registro de prueba creado únicamente para esta demostración — no se borra ningún dato clínico real.

[CAPTURA: resultado de la operación CREATE — nuevo paciente y hospitalización]

[CAPTURA: resultado de los UPDATE — campo simple y campos anidados]

[CAPTURA: resultado del SOFT DELETE — cama liberada y paciente inactivo]

[CAPTURA: resultado del HARD DELETE — conteo antes (1) y después (0) del borrado]

4. Consultas de agregación avanzadas

Las 4 consultas siguientes son nuevas frente a las ya entregadas en la Unidad 2 y en el Foro U3: aquellas se enfocaban en diagnósticos puntuales, evolución cronológica, signos vitales y conteo de pacientes por servicio. Estas 4 aprovechan específicamente las colecciones nuevas (`medicos`, `camas`) para detectar patrones operativos y clínicos distintos. Script completo: `05_consultas_avanzadas.js`.

4.1 Consulta 1 — Ocupación de camas por servicio

Pregunta que responde: ¿cuántas camas libres y ocupadas tiene cada servicio en este momento, y qué porcentaje de ocupación representa?

Etapas: `$match` → `$group` → `$project` → `$sort`.

```
db.camas.aggregate([
  { $match: { estado: { $in: ["libre", "ocupada"] } } },
  { $group: { _id: "$servicio", total_camas: { $sum: 1 },
      ocupadas: { $sum: { $cond: [{ $eq: ["$estado", "ocupada"] }, 1, 0] } } } },
  { $project: { _id: 0, servicio: "$_id", total_camas: 1, ocupadas: 1,
      libres: { $subtract: ["$total_camas", "$ocupadas"] },
      porcentaje_ocupacion: { $round: [{ $multiply: [{ $divide: ["$ocupadas", "$total_camas"] }, 100] }, 1] }
} },
  { $sort: { porcentaje_ocupacion: -1 } }
])
```

[CAPTURA: resultado de la Consulta 1 en mongosh]

4.2 Consulta 2 — Carga de hospitalizaciones por médico

Pregunta que responde: ¿qué médico tiene más hospitalizaciones a cargo (en total y activas en este momento), incluyendo a los médicos que todavía no tienen ningún paciente asignado?

Etapas: `$lookup` → `$project` → `$sort`.

```
db.medicos.aggregate([
  { $lookup: { from: "hospitalizaciones", localField: "_id",
      foreignField: "medico_id", as: "hospitalizaciones_atendidas" } },
  { $project: { _id: 0, medico: "$nombre_completo", especialidad: 1,
      total_hospitalizaciones: { $size: "$hospitalizaciones_atendidas" },
      activas_ahora: { $size: { $filter: { input: "$hospitalizaciones_atendidas",
          as: "h", cond: { $eq: ["$$h.estado", "activa"] } } } } } },
  { $sort: { total_hospitalizaciones: -1 } }
])
```

[CAPTURA: resultado de la Consulta 2 en mongosh]

4.3 Consulta 3 — Días de estancia promedio por servicio

Pregunta que responde: ¿qué servicio tiene, en promedio, las hospitalizaciones más largas? (solo se cuentan las que ya tienen alta).

Etapas: `$match` → `$project` → `$group` → `$project` → `$sort`.

```
db.hospitalizaciones.aggregate([
  { $match: { alta: { $ne: null } } },
  { $project: { servicio: 1, dias_estancia: { $dateDiff: {
        startDate: { $toDate: "$fecha_admision" },
        endDate: { $toDate: "$alta.fecha_alta" }, unit: "day" } } } },
  { $group: { _id: "$servicio", estancia_promedio_dias: { $avg: "$dias_estancia" },
      total_casos: { $sum: 1 } } },
  { $project: { _id: 0, servicio: "$_id",
      estancia_promedio_dias: { $round: ["$estancia_promedio_dias", 1] }, total_casos: 1 } },
  { $sort: { estancia_promedio_dias: -1 } }
])
```

[CAPTURA: resultado de la Consulta 3 en mongosh]

4.4 Consulta 4 — Diagnósticos más frecuentes

Pregunta que responde: ¿qué diagnósticos CIE-10 son los más comunes en la base, y en qué servicios aparecen? (patrón epidemiológico, útil para vigilancia).

Etapas: `$unwind` → `$group` → `$sort` → `$project` → `$limit`.

```
db.hospitalizaciones.aggregate([
  { $unwind: "$diagnosticos" },
  { $group: { _id: { codigo: "$diagnosticos.codigo_cie10", descripcion: "$diagnosticos.descripcion" },
      frecuencia: { $sum: 1 }, servicios: { $addToSet: "$servicio" } } },
  { $sort: { frecuencia: -1 } },
  { $project: { _id: 0, codigo_cie10: "$_id.codigo", descripcion: "$_id.descripcion",
      frecuencia: 1, servicios: 1 } },
  { $limit: 10 }
])
```

[CAPTURA: resultado de la Consulta 4 en mongosh]

5. Evidencia de uso de IA

(Contenido completo en `evidencia_ia.md`.)

Herramienta: Claude (Anthropic), modelo Claude Sonnet 5, a través de Claude Code.

Claude generó directamente el ajuste del esquema (`schema.json`), los 5 scripts de MongoDB, los datos de prueba de médicos y camas, el mapeo para vincular los datos ya existentes, las 4 consultas avanzadas y el armado de este informe. El estudiante ejecutó cada script contra la base real, corrigió junto con la IA un error real (`$round` dentro de `$avg` en un mismo `$group` no es válido en esta versión de Mongo), verificó que los resultados tuvieran sentido clínico y operativo, y es responsable de la reflexión ética final.

[CAPTURA: fragmento de la conversación con Claude mostrando el diseño de medicos/camas]

6. Reflexión ética

(Texto completo en `reflexion_etica.md`.)

La reflexión aborda tres ideas centrales: (1) que los datos de los médicos también son sensibles y requieren control de acceso, no solo los del paciente; (2) que la denormalización controlada en `camas` (campo `hospitalizacion_actual_id`) traslada la responsabilidad de integridad a la aplicación, a diferencia de una llave foránea relacional; y (3) que la flexibilidad de esquema de MongoDB (agregar campos sin `ALTER TABLE`) es una ventaja de agilidad pero un riesgo de cumplimiento si no se compensa con validadores completos, control de acceso por colección y auditoría — a diferencia del modelo relacional de la HCE Básica, donde un cambio de esquema queda declarado y es auditable por diseño.

7. Conclusiones

- El modelo documental de la Unidad 2 se pudo ampliar sin romper nada de lo ya entregado, gracias a que MongoDB no exige declarar todos los campos por adelantado — aunque esa misma característica es, a la vez, el riesgo ético discutido en la sección 6.
- Referenciar `medicos` y `camas` (en vez de embeberlos) fue la decisión correcta porque ambas entidades se reutilizan constantemente desde múltiples documentos.
- Las 4 consultas avanzadas muestran que, con pocas colecciones adicionales, se pueden responder preguntas operativas reales de un hospital: ocupación de camas, carga médica, tiempos de estancia y patrones de diagnóstico.

Video explicativo

Guion y checklist completo en `guion_video.md`. Enlace del video: *[PENDIENTE — pegar aquí el enlace una vez grabado y subido]*.

Repositorio digital

[PENDIENTE — pegar aquí el enlace del repositorio de GitHub una vez subido].

Informe generado con apoyo de IA (Claude, Anthropic) — ver sección 5, Evidencia de uso de IA.